In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

In [24…
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Input
from tensorflow.keras.utils import to_categorical
```

In [3]:
```
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

In [4]:
```
df = pd.read_csv('mnist_test.csv')
df
```

Out[4]:

	label	1x1	1x2	1x3	1x4	1x5	1x6	1x7	1x8	1x9	...	28x19	28x20	28x21	28x22
0	7	0	0	0	0	0	0	0	0	0	...	0	0	0	0
1	2	0	0	0	0	0	0	0	0	0	...	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
4	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9995	2	0	0	0	0	0	0	0	0	0	...	0	0	0	0
9996	3	0	0	0	0	0	0	0	0	0	...	0	0	0	0
9997	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0
9998	5	0	0	0	0	0	0	0	0	0	...	0	0	0	0
9999	6	0	0	0	0	0	0	0	0	0	...	0	0	0	0

10000 rows × 785 columns

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
label    0
1x1      0
1x2      0
1x3      0
1x4      0
        ..
28x24    0
28x25    0
28x26    0
28x27    0
28x28    0
Length: 785, dtype: int64
```

In [7]:
```
df.columns
```

Out[7]:
```
Index(['label', '1x1', '1x2', '1x3', '1x4', '1x5', '1x6', '1x7', '1x8',
'1x9',
       ...
       '28x19', '28x20', '28x21', '28x22', '28x23', '28x24', '28x25', '2
8x26',
       '28x27', '28x28'],
      dtype='object', length=785)
```

In [8]:
```
x = df.drop(['label'], axis=1)
y = df['label']
```

In [10…
```
x = x / 255.0
```

In [11…
```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

In [25…
```
model = Sequential([
    Input(shape=(784,)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

In [32…
```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)
```

In [33…
```
model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	
dense_8 (Dense)	(None, 128)	
dense_9 (Dense)	(None, 10)	

Total params: 101,770 (397.54 KB)

Trainable params: 101,770 (397.54 KB)

Non-trainable params: 0 (0.00 B)

In [34…
```
history = model.fit(
    x_train, y_train,
    epochs=10, batch_size=128,
    validation_split=0.1
)
```

```
Epoch 1/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 2s 13ms/step - accuracy: 0.7682 - loss: 0.9050
- val_accuracy: 0.8925 - val_loss: 0.4174
Epoch 2/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9015 - loss: 0.3503
- val_accuracy: 0.9237 - val_loss: 0.3011
Epoch 3/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9239 - loss: 0.2706
- val_accuracy: 0.9237 - val_loss: 0.2740
Epoch 4/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9371 - loss: 0.2295
- val_accuracy: 0.9400 - val_loss: 0.2373
Epoch 5/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9429 - loss: 0.1967
- val_accuracy: 0.9425 - val_loss: 0.2249
Epoch 6/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9532 - loss: 0.1684
- val_accuracy: 0.9463 - val_loss: 0.2126
Epoch 7/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9590 - loss: 0.1464
- val_accuracy: 0.9550 - val_loss: 0.1959
Epoch 8/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - accuracy: 0.9633 - loss: 0.1275
- val_accuracy: 0.9488 - val_loss: 0.1951
Epoch 9/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9679 - loss: 0.1128
- val_accuracy: 0.9488 - val_loss: 0.1788
Epoch 10/10
57/57 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9731 - loss: 0.0982
- val_accuracy: 0.9525 - val_loss: 0.1847
```

In [35…

```python
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy * 100, "%")
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9360 - loss: 0.2188
Test Loss: 0.21877557039260864
Test Accuracy: 93.59999895095825 %
```

In [50…

```python
pred = model.predict(x_test)

pred_digit = np.argmax(pred[0])
act_digit = np.argmax(y_test[0])

print("Actual Digit :", act_digit)
print("Predicted Digit :", pred_digit)
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Actual Digit : 0
Predicted Digit : 6
```

In [59…

```python
pred = model.predict(x_test)

pred_digit = np.argmax(pred[0])
```

```
act_digit = np.argmax(y_test[0])

print("Actual Digit :", act_digit)
print("Predicted Digit :", pred_digit)

plt.figure(figsize=(4, 4))

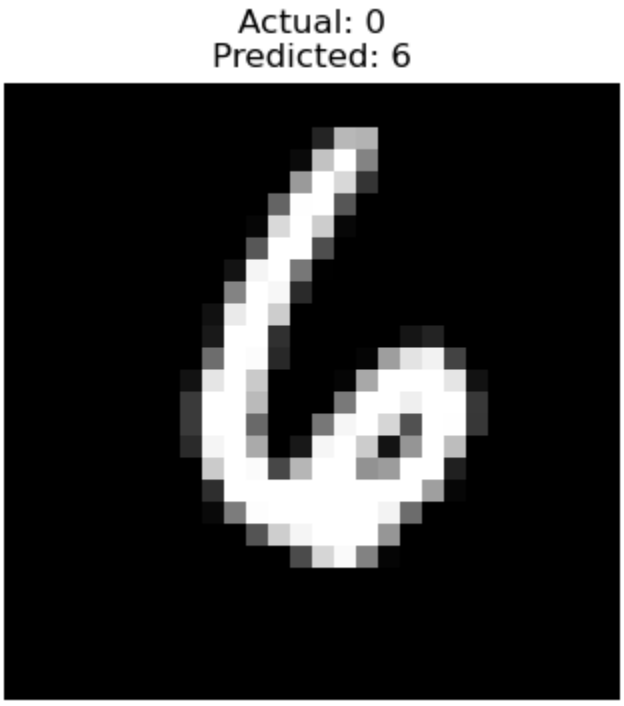
image = x_test.iloc[0].values.reshape(28, 28)

plt.imshow(image, cmap='gray')

plt.title(
    f"Actual: {act_digit}\n"
    f"Predicted: {pred_digit}"
)

plt.axis('off')
plt.show()
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
Actual Digit : 0
Predicted Digit : 6
```

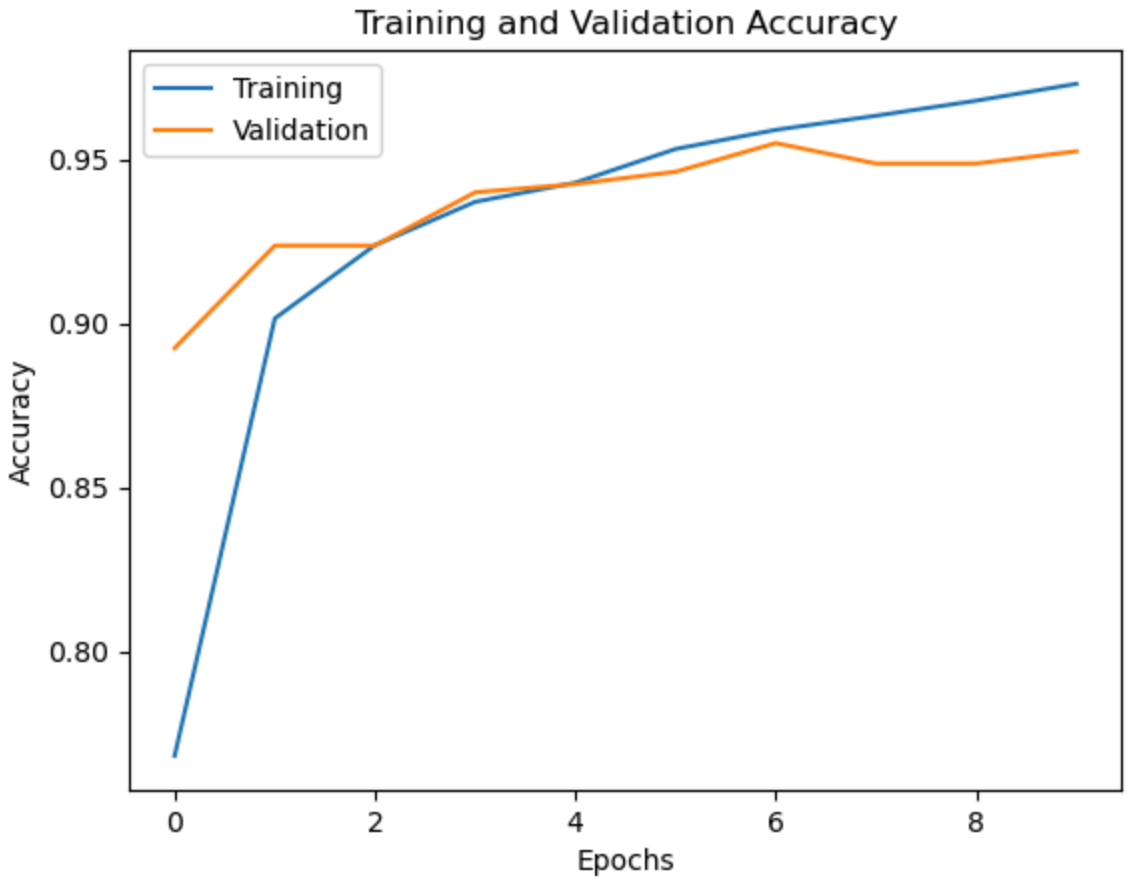


In [46…

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('Training and Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')

plt.legend(['Training', 'Validation'])
plt.show()
```

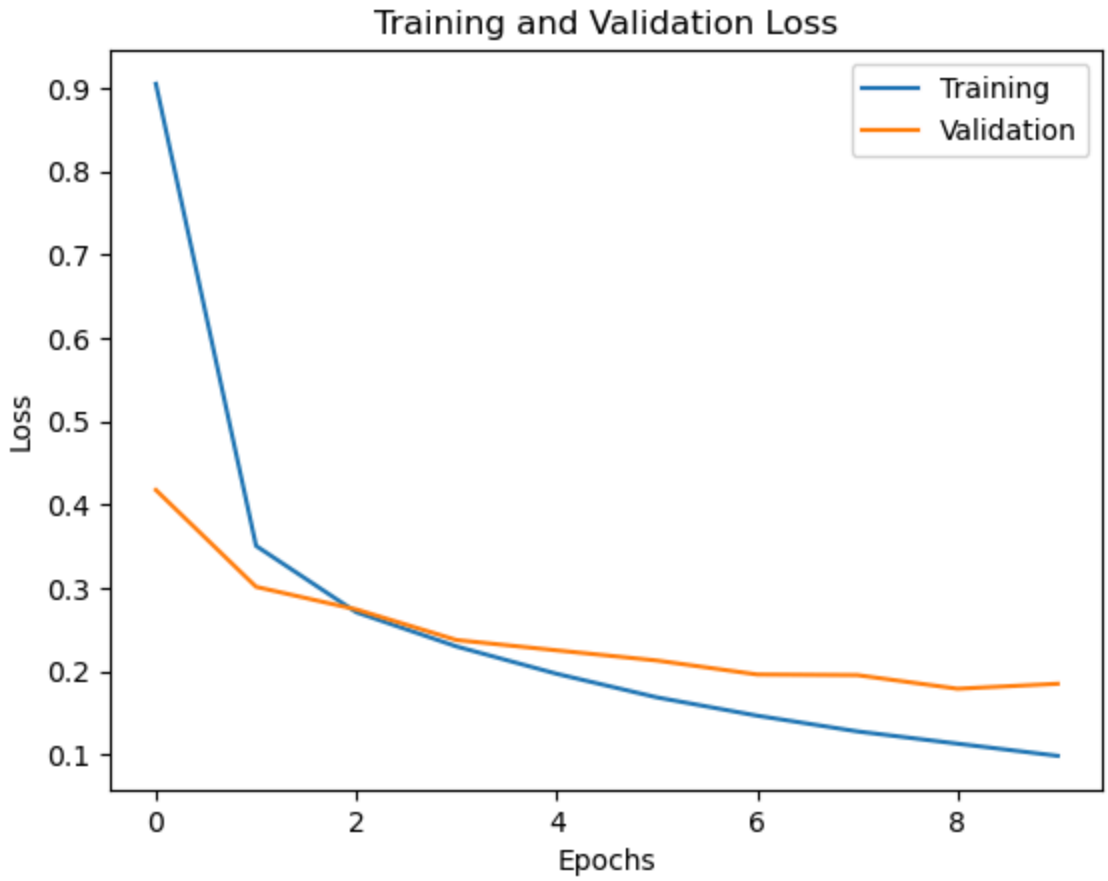


In [47…

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')

plt.legend(['Training', 'Validation'])
plt.show()
```


Training and Validation Loss
Training
Validation
Loss
Epochs
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8

In [ ]: